NOMBRE: ALEJANDRO JOSE GOMEZ HERNANDEZ

CARNE: 20347

FECHA:6/10/24

Repositorio: https://github.com/aleg001/S2_NLP_2024

Objetivo: La idea de esta práctica es que puedan llevar a cabo una tarea de Name Entity Recognition (NER) con Bert y spantBert, y sean capaces de comparar cuál rinde mejor.

Responda las siguiente preguntas:

1. ¿Qué es una tarea ner?

Básicamente es una tarea para identificar y clasificar palabras/frases en un texto según su categoría o entidad. Podrían ser organizaciones, lugares, personas, entre otras. Puede ser útil en donde se quiera tener comprensión estructurada, como para indexar búsquedas, o haceer chatbots.

2. ¿Cuál metrica de rendimiento debe usar en este caso?

La métrica de rendimiento típica para tarea ner es F1-Score. Donde se equilibra precisión y que sea exahustivo. Para poder clasificar correctamente ya que acá los falsos-positivos/falsos-negativos si son importantes.

```
!pip install torch transformers datasets scikit-learn evaluate
```

```
Requirement already satisfied: filelock in /usr/local/lib/python3.10/dist-packages (from torch) (3.16.1)
Requirement already satisfied: typing-extensions>=4.8.0 in /usr/local/lib/python3.10/dist-packages (from torch) (4.12.2)
Requirement already satisfied: sympy in /usr/local/lib/python3.10/dist-packages (from torch) (1.13.3)
Requirement already satisfied: networkx in /usr/local/lib/python3.10/dist-packages (from torch) (3.3)
Requirement already satisfied: jinja2 in /usr/local/lib/python3.10/dist-packages (from torch) (3.1.4)
Requirement already satisfied: fsspec in /usr/local/lib/python3.10/dist-packages (from torch) (2024.6.1)
Requirement already satisfied: huggingface-hub<1.0,>=0.23.2 in /usr/local/lib/python3.10/dist-packages (from transformers) (0.24.7)
Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.10/dist-packages (from transformers) (1.26.4)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.10/dist-packages (from transformers) (24.1)
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.10/dist-packages (from transformers) (6.0.2)
Requirement already satisfied: regex!=2019.12.17 in /usr/local/lib/python3.10/dist-packages (from transformers) (2024.9.11)
Requirement already satisfied: requests in /usr/local/lib/python3.10/dist-packages (from transformers) (2.32.3)
Requirement already satisfied: safetensors>=0.4.1 in /usr/local/lib/python3.10/dist-packages (from transformers) (0.4.5)
Requirement already satisfied: tokenizers<0.20,>=0.19 in /usr/local/lib/python3.10/dist-packages (from transformers) (0.19.1)
Requirement already satisfied: tqdm>=4.27 in /usr/local/lib/python3.10/dist-packages (from transformers) (4.66.5)
Requirement already satisfied: pyarrow>=15.0.0 in /usr/local/lib/python3.10/dist-packages (from datasets) (16.1.0)
Collecting dill<0.3.9,>=0.3.0 (from datasets)
  Downloading dill-0.3.8-py3-none-any.whl.metadata (10 kB)
Requirement already satisfied: pandas in /usr/local/lib/python3.10/dist-packages (from datasets) (2.2.2)
Collecting xxhash (from datasets)
  Downloading xxhash-3.5.0-cp310-cp310-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (12 kB)
Collecting multiprocess (from datasets)
  Downloading multiprocess-0.70.17-py310-none-any.whl.metadata (7.2 kB)
Requirement already satisfied: aiohttp in /usr/local/lib/python3.10/dist-packages (from datasets) (3.10.8)
Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.10/dist-packages (from scikit-learn) (1.13.1)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.10/dist-packages (from scikit-learn) (1.4.2)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.10/dist-packages (from scikit-learn) (3.5.0)
Requirement already satisfied: aiohappyeyeballs>=2.3.0 in /usr/local/lib/python3.10/dist-packages (from aiohttp->datasets) (2.4.3)
Requirement already satisfied: aiosignal>=1.1.2 in /usr/local/lib/python3.10/dist-packages (from aiohttp->datasets) (1.3.1)
Requirement already satisfied: attrs>=17.3.0 in /usr/local/lib/python3.10/dist-packages (from aiohttp->datasets) (24.2.0)
Requirement already satisfied: frozenlist>=1.1.1 in /usr/local/lib/python3.10/dist-packages (from aiohttp->datasets) (1.4.1)
Requirement already satisfied: multidict<7.0,>=4.5 in /usr/local/lib/python3.10/dist-packages (from aiohttp->datasets) (6.1.0)
Requirement already satisfied: yarl<2.0,>=1.12.0 in /usr/local/lib/python3.10/dist-packages (from aiohttp->datasets) (1.13.1)
Requirement already satisfied: async-timeout<5.0,>=4.0 in /usr/local/lib/python3.10/dist-packages (from aiohttp->datasets) (4.0.3)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.10/dist-packages (from requests->transformers) (3.3.2)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.10/dist-packages (from requests->transformers) (3.10)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.10/dist-packages (from requests->transformers) (2.2.3)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.10/dist-packages (from requests->transformers) (2024.8.30)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.10/dist-packages (from jinja2->torch) (2.1.5)
INFO: pip is looking at multiple versions of multiprocess to determine which version is compatible with other requirements. This could take a while.
  Downloading multiprocess-0.70.16-py310-none-any.whl.metadata (7.2 kB)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.10/dist-packages (from pandas->datasets) (2.8.2)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.10/dist-packages (from pandas->datasets) (2024.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.10/dist-packages (from pandas->datasets) (2024.2)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.10/dist-packages (from sympy->torch) (1.3.0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.10/dist-packages (from python-dateutil>=2.8.2->pandas->datasets) (1.16.0)
Downloading datasets-3.0.1-py3-none-any.whl (471 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 471.6/471.6 kB 5.0 MB/s eta 0:00:00
Downloading evaluate-0.4.3-py3-none-any.whl (84 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 84.0/84.0 kB 4.1 MB/s eta 0:00:00
Downloading dill-0.3.8-py3-none-any.whl (116 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 116.3/116.3 kB 3.3 MB/s eta 0:00:00
Downloading multiprocess-0.70.16-py310-none-any.whl (134 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 134.8/134.8 kB 3.8 MB/s eta 0:00:00
Downloading xxhash-3.5.0-cp310-cp310-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (194 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 194.1/194.1 kB 6.9 MB/s eta 0:00:00
Installing collected packages: xxhash, dill, multiprocess, datasets, evaluate
Successfully installed datasets-3.0.1 dill-0.3.8 evaluate-0.4.3 multiprocess-0.70.16 xxhash-3.5.0
```

```
#Librerías
import torch
from transformers import AutoTokenizer, AutoModelForTokenClassification, TrainingArguments, Trainer
from datasets import load_dataset, load_metric
```

```
5 import numpy as np
6 from sklearn.metrics import classification_report
```

```
---------------------------------------------------------------------------
ImportError                               Traceback (most recent call last)
<ipython-input-3-8c7a75d9e859> in <cell line: 4>()
      2 import torch
      3 from transformers import AutoTokenizer, AutoModelForTokenClassification, TrainingArguments, Trainer
----> 4 from datasets import load_dataset, load_metric
      5 import numpy as np
      6 from sklearn.metrics import classification_report

ImportError: cannot import name 'load_metric' from 'datasets' (/usr/local/lib/python3.10/dist-packages/datasets/__init__.py)

---------------------------------------------------------------------------
NOTE: If your import is failing due to a missing package, you can
manually install dependencies using either !pip or !apt.

To view examples of installing some common dependencies, click the
"Open Examples" button below.
---------------------------------------------------------------------------
```

OPEN EXAMPLES

Pasos siguientes: Explicar error

```
1 import torch
2 from transformers import AutoTokenizer, AutoModelForTokenClassification, TrainingArguments, Trainer
3 from datasets import load_dataset
4 import evaluate
5 import numpy as np
6 from sklearn.metrics import classification_report
```

```
1 #Dataset
2 dataset = load_dataset("conll2003")
```

```
/usr/local/lib/python3.10/dist-packages/huggingface_hub/utils/_token.py:89: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it as secret in your Google Colab and restart your session.
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
conll2003.py: 100% 9.57k/9.57k [00:00<00:00, 119kB/s]
README.md: 100% 12.3k/12.3k [00:00<00:00, 147kB/s]
The repository for conll2003 contains custom code which must be executed to correctly load the dataset. You can inspect the repository content at https://hf.co/datasets/conll2003.
You can avoid this prompt in future by passing the argument `trust_remote_code=True`.

Do you wish to run the custom code? [y/N] y
Downloading data: 100% 983k/983k [00:00<00:00, 1.44MB/s]
Generating train split: 100% 14041/14041 [00:10<00:00, 1205.39 examples/s]
Generating validation split: 100% 3250/3250 [00:02<00:00, 1363.74 examples/s]
Generating test split: 100% 3453/3453 [00:02<00:00, 1832.04 examples/s]
```

```
1 #Modelos
2 spanbert = "SpanBERT/spanbert-base-cased"
3 bert = "bert-base-uncased"
```

```
1 f1_metric = evaluate.load("f1")
```

```
Downloading builder script: 100% 6.77k/6.77k [00:00<00:00, 97.2kB/s]
```

```
1 spanbert_tokenizer = AutoTokenizer.from_pretrained(spanbert)
2 bert_tokenizer = AutoTokenizer.from_pretrained(bert)
```

```
config.json: 100% 413/413 [00:00<00:00, 4.19kB/s]
vocab.txt: 100% 213k/213k [00:00<00:00, 3.42MB/s]
/usr/local/lib/python3.10/dist-packages/transformers/tokenization_utils_base.py:1601: FutureWarning: `clean_up_tokenization_spaces` was not set. It will be set to `True` by default. This behavior will be depracted in transformers v4.45, and will be then
  warnings.warn(
tokenizer_config.json: 100% 48.0/48.0 [00:00<00:00, 905B/s]
config.json: 100% 570/570 [00:00<00:00, 7.87kB/s]
vocab.txt: 100% 232k/232k [00:00<00:00, 3.02MB/s]
tokenizer.json: 100% 466k/466k [00:00<00:00, 5.05MB/s]
```

```
spanbert_model = AutoModelForTokenClassification.from_pretrained(spanbert)
bert_model = AutoModelForTokenClassification.from_pretrained(bert)
```

```
pytorch_model.bin: 100% 215M/215M [00:02<00:00, 122MB/s]
Some weights of BertForTokenClassification were not initialized from the model checkpoint at SpanBERT/spanbert-base-cased and are newly initialized: ['classifier.bias', 'classifier.weight']
You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.
model.safetensors: 100% 440M/440M [00:04<00:00, 129MB/s]
Some weights of BertForTokenClassification were not initialized from the model checkpoint at bert-base-uncased and are newly initialized: ['classifier.bias', 'classifier.weight']
You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.
```

```
assert num_labels == 9, "The number of labels should match the dataset's label range."
```

```
for split in ["train", "validation", "test"]:
    all_labels = [label for example in dataset[split]["ner_tags"] for label in example]
    out_of_range_labels = [label for label in all_labels if label < 0 or label >= num_labels]
    print(f"{split} out-of-range labels: {out_of_range_labels}")
```

```
train out-of-range labels: []
validation out-of-range labels: []
test out-of-range labels: []
```

```
class CustomTrainer(Trainer):
    def compute_loss(self, model, inputs, return_outputs=False):
        labels = inputs.pop("labels")
        outputs = model(**inputs)
        logits = outputs.logits
        loss_fct = nn.CrossEntropyLoss(ignore_index=-100)
        loss = loss_fct(logits.view(-1, self.model.config.num_labels), labels.view(-1))
        return (loss, outputs) if return_outputs else loss

```

```
label_list = dataset['train'].features['ner_tags'].feature.names
num_labels = len(label_list)
def tokenize_and_align_labels(examples, tokenizer):
    tokenized_inputs = tokenizer(examples["tokens"], truncation=True, is_split_into_words=True)
    labels = []
    for i, label in enumerate(examples["ner_tags"]):
        word_ids = tokenized_inputs.word_ids(batch_index=i)
        label_ids = []
        for word_idx in word_ids:
            if word_idx is None or label[word_idx] >= num_labels:
                label_ids.append(-100)
            else:
                label_ids.append(label[word_idx])
        labels.append(label_ids)
    tokenized_inputs["labels"] = labels
    return tokenized_inputs

tokenized_spanbert = dataset.map(lambda x: tokenize_and_align_labels(x, spanbert_tokenizer), batched=True)
tokenized_bert = dataset.map(lambda x: tokenize_and_align_labels(x, bert_tokenizer), batched=True)

```

```
def metricas(eval_pred):
    logits, labels = eval_pred
    predictions = np.argmax(logits, axis=-1)
    true_labels = [[label for label in labels[i] if label != -100] for i in range(len(labels))]
    true_predictions = [[p for (p, l) in zip(predictions[i], labels[i]) if l != -100] for i in range(len(labels))]
    return f1_metric.compute(predictions=true_predictions, references=true_labels)

```

```
from transformers import DataCollatorForTokenClassification
data_collator = DataCollatorForTokenClassification(tokenizer=spanbert_tokenizer)

data_collator_spanbert = DataCollatorForTokenClassification(tokenizer=spanbert_tokenizer)
data_collator_bert = DataCollatorForTokenClassification(tokenizer=bert_tokenizer)

```

```
# Argumentos de entrenamiento
training_args = TrainingArguments(
    output_dir="./results",
    evaluation_strategy="epoch",
    per_device_train_batch_size=8,
    per_device_eval_batch_size=8,
    num_train_epochs=1,
    logging_steps=10,
    save_strategy="epoch",
    ignore_data_skip=True
)
```

```
print("Label List:", label_list)
print("Number of Labels:", num_labels)
```

```
Label List: ['O', 'B-PER', 'I-PER', 'B-ORG', 'I-ORG', 'B-LOC', 'I-LOC', 'B-MISC', 'I-MISC']
Number of Labels: 9
```

```
trainer_spanbert = CustomTrainer(
    model=spanbert_model,
    args=training_args,
    train_dataset=tokenized_spanbert["train"],
    eval_dataset=tokenized_spanbert["validation"],
    data_collator=data_collator_spanbert,
    compute_metrics=metricas
)
```

```
import torch
import torch.nn as nn
```

```
spanbert_results = trainer_spanbert.evaluate()
```

```
---------------------------------------------------------------------------
IndexError                                Traceback (most recent call last)
<ipython-input-55-f29d9be8da67> in <cell line: 1>()
----> 1 spanbert_results = trainer_spanbert.evaluate()

                                 7 frames
/usr/local/lib/python3.10/dist-packages/torch/nn/functional.py in cross_entropy(input, target, weight, size_average, ignore_index, reduce, reduction, label_smoothing)
   3102     if size_average is not None or reduce is not None:
   3103         reduction = _Reduction.legacy_get_string(size_average, reduce)
-> 3104     return torch._C._nn.cross_entropy_loss(input, target, weight, _Reduction.get_enum(reduction), ignore_index, label_smoothing)
   3105
   3106

IndexError: Target 3 is out of bounds.
```

Pasos siguientes: Explicar error

```
trainer_spanbert.train()
```

```
---------------------------------------------------------------------------
IndexError                                Traceback (most recent call last)
<ipython-input-52-83b96be6df96> in <cell line: 1>()
----> 1 trainer_spanbert.train()

                                 7 frames
/usr/local/lib/python3.10/dist-packages/torch/nn/functional.py in cross_entropy(input, target, weight, size_average, ignore_index, reduce, reduction, label_smoothing)
   3102     if size_average is not None or reduce is not None:
   3103         reduction = _Reduction.legacy_get_string(size_average, reduce)
-> 3104     return torch._C._nn.cross_entropy_loss(input, target, weight, _Reduction.get_enum(reduction), ignore_index, label_smoothing)
   3105
   3106

IndexError: Target 7 is out of bounds.
```

Pasos siguientes: Explicar error

```
trainer_spanbert.train()
spanbert_results = trainer_spanbert.evaluate()
```

```
---------------------------------------------------------------------------
IndexError                                Traceback (most recent call last)
<ipython-input-25-082ca1663674> in <cell line: 1>()
----> 1 trainer_spanbert.train()
      2 spanbert_results = trainer_spanbert.evaluate()

                                 10 frames
/usr/local/lib/python3.10/dist-packages/torch/nn/functional.py in cross_entropy(input, target, weight, size_average, ignore_index, reduce, reduction, label_smoothing)
   3102     if size_average is not None or reduce is not None:
   3103         reduction = _Reduction.legacy_get_string(size_average, reduce)
-> 3104     return torch._C._nn.cross_entropy_loss(input, target, weight, _Reduction.get_enum(reduction), ignore_index, label_smoothing)
   3105
   3106

IndexError: Target 7 is out of bounds.
```

Pasos siguientes: Explicar error

```
model_names = ["SpanBERT/spanbert-base-cased", "bert-base-uncased"]
tokenizers = [AutoTokenizer.from_pretrained(name) for name in model_names]
models = [AutoModelForTokenClassification.from_pretrained(name, num_labels=num_labels) for name in model_names]

```

```
Some weights of BertForTokenClassification were not initialized from the model checkpoint at SpanBERT/spanbert-base-cased and are newly initialized: ['classifier.bias', 'classifier.weight']
You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.
Some weights of BertForTokenClassification were not initialized from the model checkpoint at bert-base-uncased and are newly initialized: ['classifier.bias', 'classifier.weight']
You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.
```

```python
def tokenize_and_align_labels(examples, tokenizer):
    tokenized_inputs = tokenizer(examples["tokens"], truncation=True, is_split_into_words=True)
    labels = []
    for i, label in enumerate(examples["ner_tags"]):
        word_ids = tokenized_inputs.word_ids(batch_index=i)
        label_ids = [-100 if word_idx is None or label[word_idx] >= num_labels else label[word_idx] for word_idx in word_ids]
        labels.append(label_ids)
    tokenized_inputs["labels"] = labels
    return tokenized_inputs
```

```python
tokenized_datasets = []
for tokenizer in tokenizers:
    tokenized_datasets.append(dataset.map(lambda x: tokenize_and_align_labels(x, tokenizer), batched=True))

```

```
Map: 100% 14041/14041 [00:04<00:00, 2779.23 examples/s]
Asking to truncate to max_length but no maximum length is provided and the model has no predefined maximum length. Default to no truncation.
Map: 100% 3250/3250 [00:01<00:00, 2701.74 examples/s]
Map: 100% 3453/3453 [00:01<00:00, 2974.25 examples/s]
Map: 100% 14041/14041 [00:06<00:00, 1859.54 examples/s]
Map: 100% 3250/3250 [00:01<00:00, 3088.15 examples/s]
Map: 100% 3453/3453 [00:01<00:00, 3210.80 examples/s]
```

```python
def compute_f1(eval_pred):
    predictions, labels = eval_pred
    predictions = np.argmax(predictions, axis=-1)
    true_labels = [[label for label in labels[i] if label != -100] for i in range(len(labels))]
    true_predictions = [[pred for (pred, label) in zip(predictions[i], labels[i]) if label != -100] for i in range(len(labels))]
    return f1_metric.compute(predictions=true_predictions, references=true_labels)

```

```python
def train_and_evaluate_model(model, tokenizer, tokenized_dataset):
    # Argumentos de entrenamiento
    training_args = TrainingArguments(
        output_dir="./results",
        evaluation_strategy="epoch",
        per_device_train_batch_size=8,
        per_device_eval_batch_size=8,
        num_train_epochs=1,
        logging_steps=10,
    )

    # Configurar Trainer
    trainer = Trainer(
        model=model,
        args=training_args,
        train_dataset=tokenized_dataset["train"],
        eval_dataset=tokenized_dataset["validation"],
        data_collator=DataCollatorForTokenClassification(tokenizer=tokenizer),
        compute_metrics=compute_f1
    )

    # Entrenar y evaluar
    trainer.train()
    results = trainer.evaluate()
    return results["eval_f1"]
```

```python
f1_scores = []
for i, (model, tokenizer, tokenized_dataset) in enumerate(zip(models, tokenizers, tokenized_datasets)):
    f1_score = train_and_evaluate_model(model, tokenizer, tokenized_dataset)
    f1_scores.append(f1_score)
    print(f"Modelo: {model_names[i]}, F1-Score: {f1_score}")
```

```
[ 459/1756 38:19 < 1:48:45, 0.20 it/s, Epoch 0.26/1]
Epoch  Training Loss  Validation Loss
```

BERT es versátil y efectivo en NER normal, SpanBERT es mejor cuando se demanda una representación más profunda de las relaciones entre entidades. SpanBERT normalmente ofrece mejor rendimiento en NER.Justo hay varios papers que lo comprueban, como este: https://bmcmedinformdecismak.biomedcentral.com/articles/10.1186/s12911-022-01967-7

Mi modelo se sigue entrando, ya que me dio error de index al correrlo originalmente. Sin embargo, opté también por investigar. Leí que SpanBert es

BERT es versátil y efectivo en NER normal, SpanBERT es mejor cuando se demanda una representación más profunda de las relaciones entre entidades. SpanBERT normalmente ofrece mejor rendimiento en NER.Justo hay varios papers que lo comprueban, como este: https://bmcmedinformdecismak.biomedcentral.com/articles/10.1186/s12911-022-01967-7

Mi modelo se sigue entrando, ya que me dio error de index al correrlo originalmente. Sin embargo, opté también por investigar. Leí que SpanBert es útil para temas biomédicos y jurídicos. Lo cuál me parece muy interesante y bastante útil.

Subiré al repositorio este mismo archivo pero ya actualizado cuando termine de entrenar y tenga resultados.

Mi modelo se sigue entrando, ya que me dio error de index al correrlo originalmente. Sin embargo, opté también por investigar. Leí que SpanBert es útil para temas biomédicos y jurídicos. Lo cuál me parece muy interesante y bastante útil.

Subiré al repositorio este mismo archivo pero ya actualizado cuando termine de entrenar y tenga resultados.